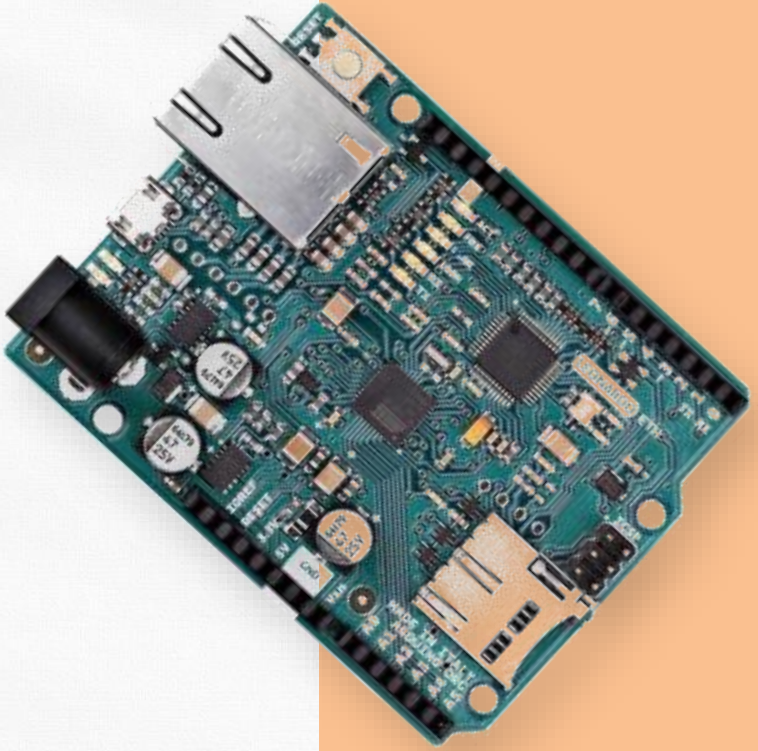
单片机



第5章：外部中断

目 录

CONTENTS

01. 中断的概念和作用

02. STM32的中断系统

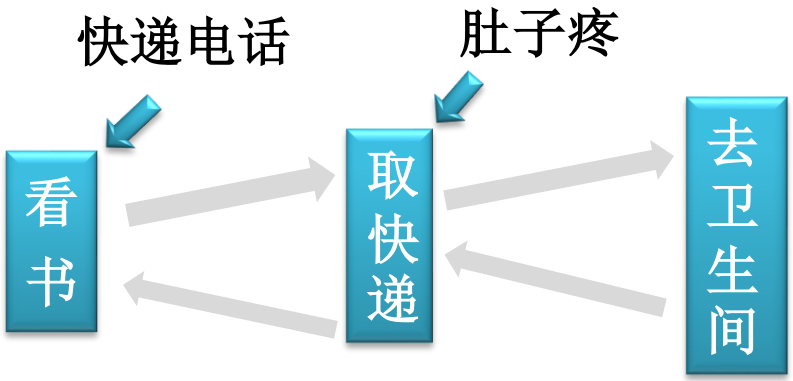
03. 外部中断EXTI

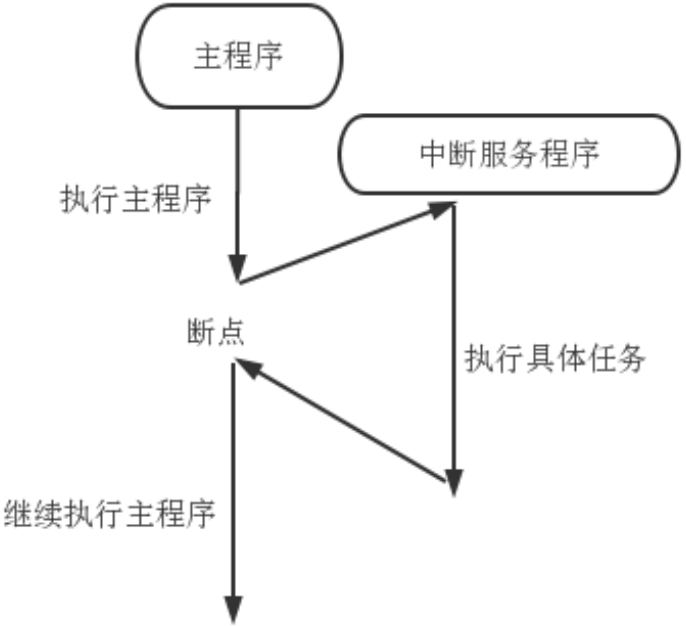
04. EXTI模块的HAL库接口函数及应用

01

中断的概念和作用

什么是中断？





中断：在处理器执行程序的过程中，被处理器内部或外部事件所打断，暂停当前程序的执行，转而去执行该事件对应的处理程序的过程。

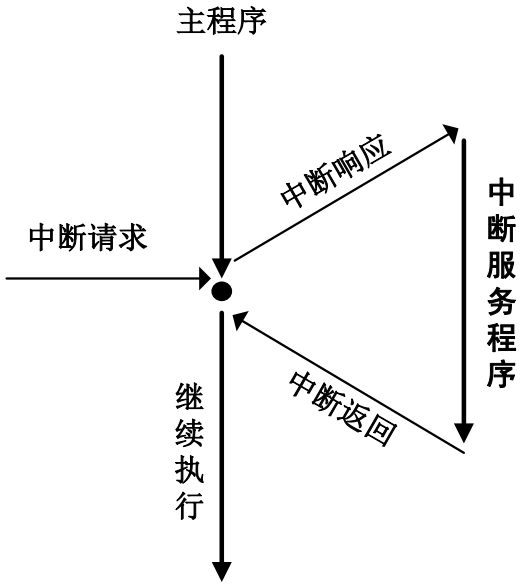
什么是中断？

在中断过程中，引发中断的事件称为**中断源**；当前正在执行的程序称为**主程序**；主程序被暂停的位置称为**断点**；事件所对应的处理程序称为**中断服务程序**（interrupt service routine，ISR）

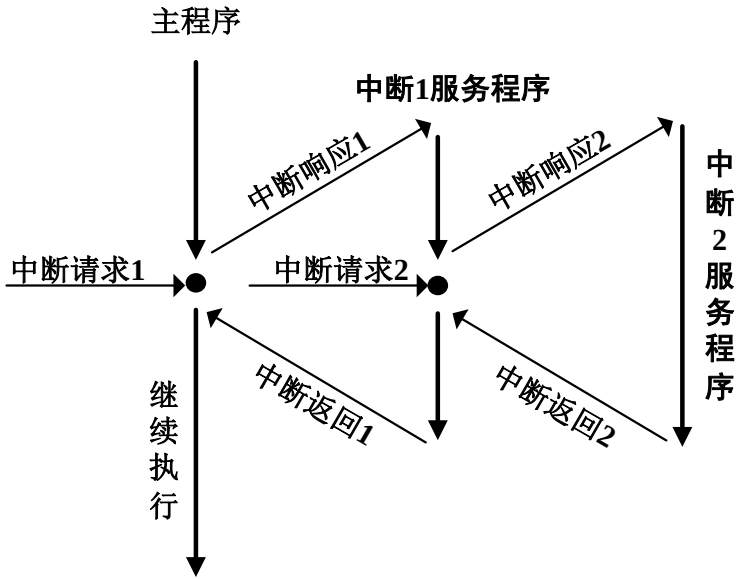
看书	主程序
快递电话、肚子疼	中断源
取快递、去卫生间	中断服务程序

中断处理流程

中断处理流程：中断请求、中断响应、中断服务和中断返回



单重中断的中断处理流程



多重中断的中断处理流程

中断处理流程

中断请求

中断请求是中断源向CPU发出中断请求信号，此时中断控制系统的中断请求寄存器被置位，向CPU请求中断。

中断响应

CPU的中断系统判断中断源的中断请求是否符合中断响应条件，如果符合条件，则暂时中断当前程序并控制程序跳转到中断服务程序。

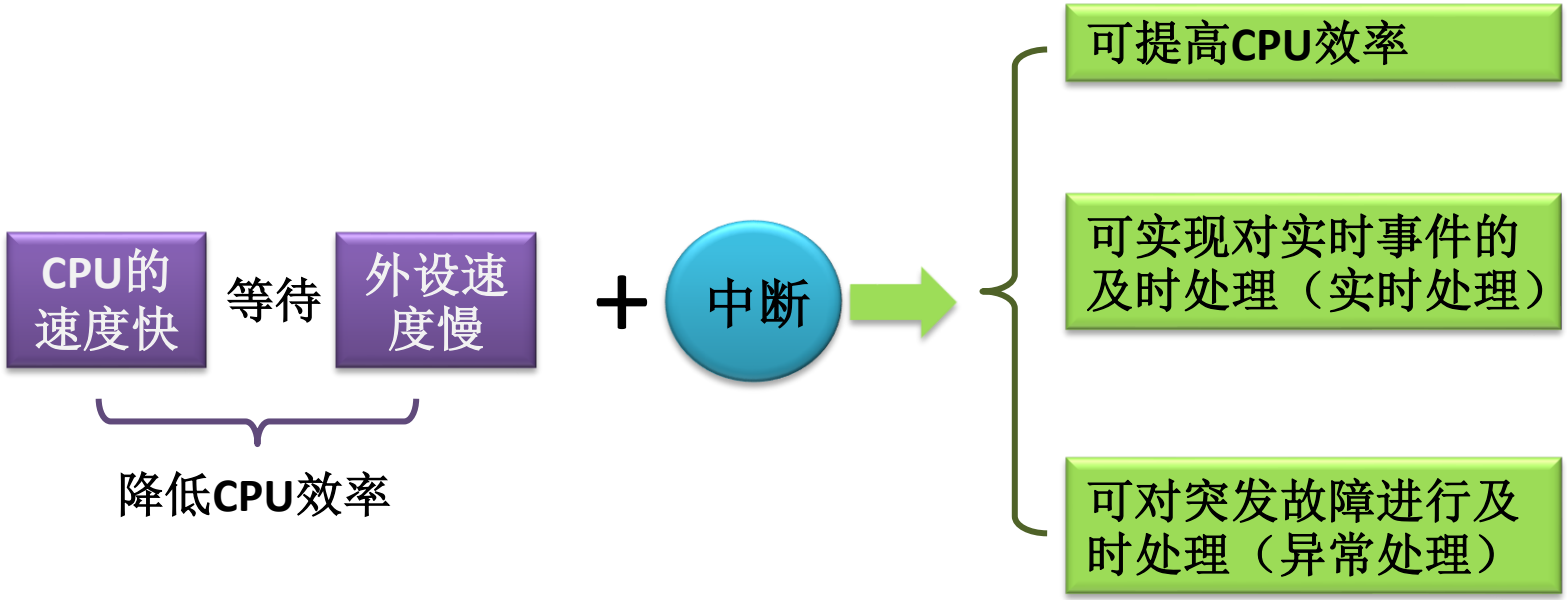
中断服务

为处理中断而编写的程序称为中断服务程序，是由开发人员针对具体中断所要实现的功能进行设计和编写的，需要由开发人员来实现。

中断返回

CPU退出中断服务程序，返回到中断请求响应之前被中止的位置继续执行主程序。这部分操作同样由硬件来实现，不需要开发人员进行处理。

中断的作用


CPU的速度快
等待
外设速度慢
降低CPU效率
+
中断
可提高CPU效率
可实现对实时事件的及时处理（实时处理）
可对突发故障进行及时处理（异常处理）

02

STM32的中断系统

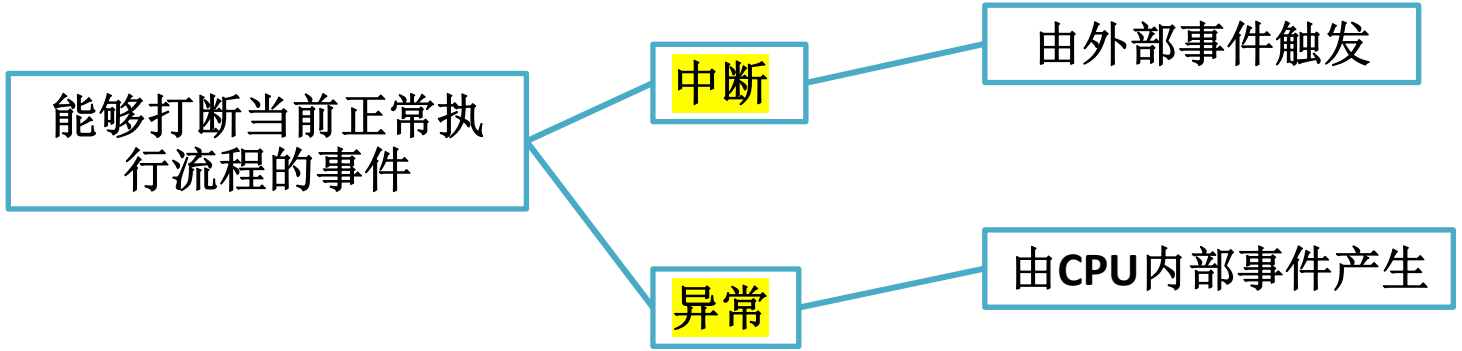
STM32中断和异常

STM32中断和异常向量表

STM32中断优先级

STM32中断服务程序

STM32中断和异常向量表



中断可以由硬件和软件触发。异常通常是由内核自身产生的，多由软件触发，比如除法出错异常、预取指失败异常等；而中断是由内核外部产生的，一般由硬件触发，比如定时器中断、串口中断等。

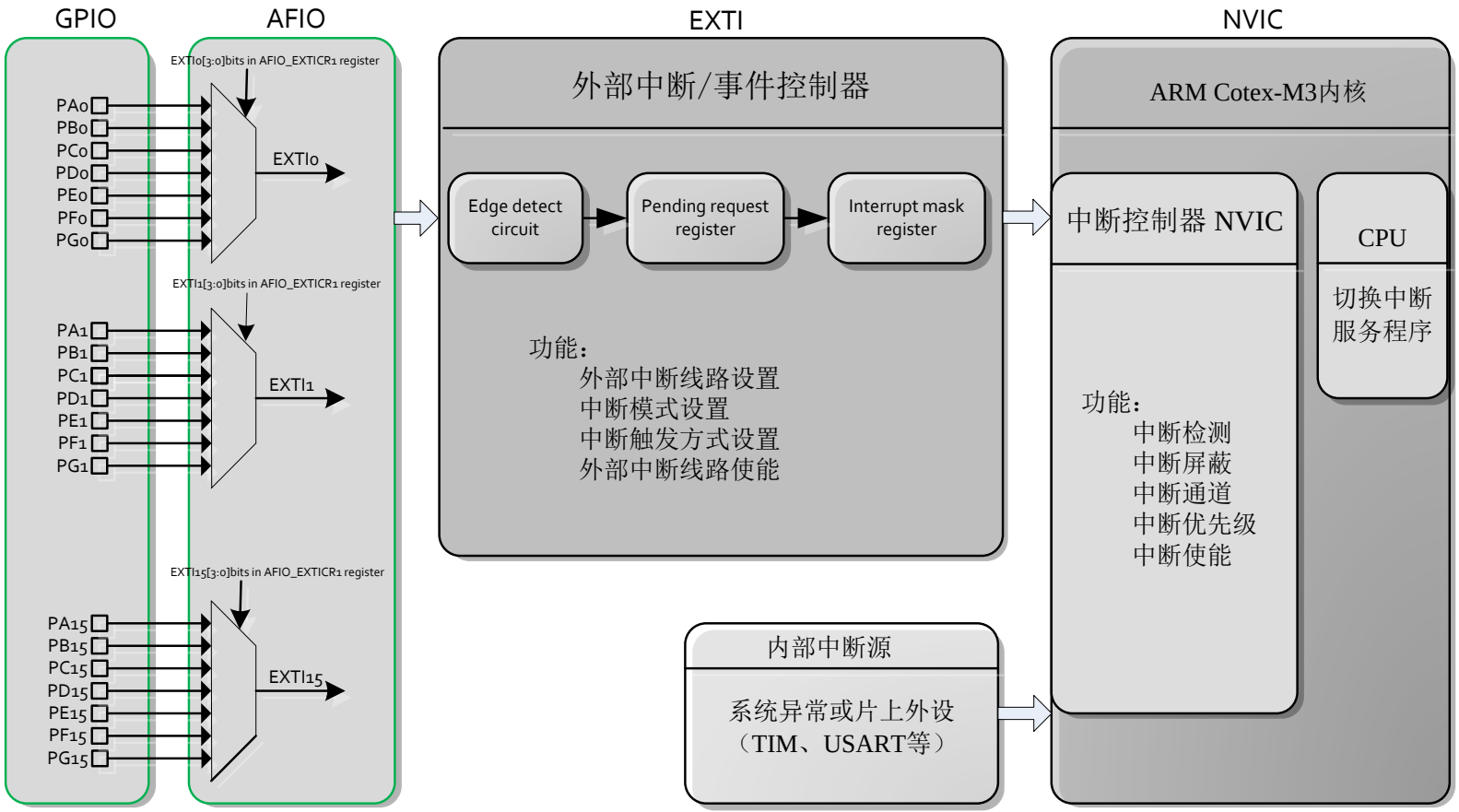
位置	优先级	优先级类型	名称	说明	地址
	-	-	-	保留	0x0000_0000
	-3	固定	Reset	复位	0x0000_0004
	-2	固定	NMI	不可屏蔽中断 RCC时钟安全系统(CSS)联接到NMI向量	0x0000_0008
	-1	固定	硬件失效(HardFault)	所有类型的失效	0x0000_000C
	0	可设置	存储管理(MemManage)	存储器管理	0x0000_0010
	1	可设置	总线错误(BusFault)	预取指失败，存储器访问失败	0x0000_0014
	2	可设置	错误应用(UsageFault)	未定义的指令或非法状态	0x0000_0018
	-	-	-	保留	0x0000_001C ~0x0000_002B
	3	可设置	SVCall	通过SWI指令的系统服务调用	0x0000_002C
	4	可设置	调试监控(DebugMonitor)	调试监控器	0x0000_0030
	-	-	-	保留	0x0000_0034
	5	可设置	PendSV	可挂起的系统服务	0x0000_0038
	6	可设置	SysTick	系统嘀嗒定时器	0x0000_003C
0	7	可设置	WWDG	窗口定时器中断	0x0000_0040
1	8	可设置	PVD	连到EXTI的电源电压检测(PVD)中断	0x0000_0044
2	9	可设置	TAMPER	侵入检测中断	0x0000_0048
3	10	可设置	RTC	实时时钟(RTC)全局中断	0x0000_004C
4	11	可设置	FLASH	闪存全局中断	0x0000_0050
5	12	可设置	RCC	复位和时钟控制(RCC)中断	0x0000_0054
6	13	可设置	EXTI0	EXTI线0中断	0x0000_0058
7	14	可设置	EXTI1	EXTI线1中断	0x0000_005C
8	15	可设置	EXTI2	EXTI线2中断	0x0000_0060
9	16	可设置	EXTI3	EXTI线3中断	0x0000_0064
10	17	可设置	EXTI4	EXTI线4中断	0x0000_0068
11	18	可设置	DMA1通道1	DMA1通道1全局中断	0x0000_006C
12	19	可设置	DMA1通道2	DMA1通道2全局中断	0x0000_0070
13	20	可设置	DMA1通道3	DMA1通道3全局中断	0x0000_0074
14	21	可设置	DMA1通道4	DMA1通道4全局中断	0x0000_0078
15	22	可设置	DMA1通道5	DMA1通道5全局中断	0x0000_007C
16	23	可设置	DMA1通道6	DMA1通道6全局中断	0x0000_0080
17	24	可设置	DMA1通道7	DMA1通道7全局中断	0x0000_0084
18	25	可设置	ADC1_2	ADC1和ADC2的全局中断	0x0000_0088
19	26	可设置	USB_HP_CAN_TX	USB高优先级或CAN发送中断	0x0000_008C
20	27	可设置	USB_LP_CAN_RX0	USB低优先级或CAN接收0中断	0x0000_0090
21	28	可设置	CAN_RX1	CAN接收1中断	0x0000_0094
22	29	可设置	CAN_SCE	CAN SCE中断	0x0000_0098
23	30	可设置	EXTI9_5	EXTI线[9:5]中断	0x0000_009C
24	31	可设置	TIM1_BRK	TIM1刹车中断	0x0000_00A0
25	32	可设置	TIM1_UP	TIM1更新中断	0x0000_00A4
26	33	可设置	TIM1_TRG_COM	TIM1触发和通信中断	0x0000_00A8
27	34	可设置	TIM1_CC	TIM1捕获比较中断	0x0000_00AC
28	35	可设置	TIM2	TIM2全局中断	0x0000_00B0
29	36	可设置	TIM3	TIM3全局中断	0x0000_00B4
30	37	可设置	TIM4	TIM4全局中断	0x0000_00B8

嵌套向量中断控制器（NVIC）

作用：用来管理全部的异常和中断。通过对NVIC的编程，可以实现中断的使能、中断优先级的设置以及中断触发方式的选择等功能。

管理手段：中断通道、中断向量表、中断服务程序

STM32的内部中断处理机制



STM32中断优先级

抢占优先级和子优先级

STM32 的中断源具有两种优先级，一种为抢占优先级，另一种为子优先级（响应优先级），其属性编号越小，表明它的优先级别越高。

抢占是指打断其它中断的属性，即低抢占优先级的中断A可以被高抢占优先级的中断B打断，执行完中断服务函数 B后 再返回继续执行中断服务函数 A，由此会出现中断嵌套。

STM32中断优先级

中断优先级判断：当多个中断源同时提出中断请求时，先判断抢占优先级的大小，如果抢占优先级相同，则比较响应优先级的大小，若抢占优先级和响应优先级均相同，则根据中断向量表中的顺序来决定；

中断源	抢占优先级	子优先级
A	0	0
B	1	0
C	1	1

根据A、B、C三个中断的优先级设置，分析三个中断执行情况。

STM32中断优先级分组

中断源通过中断通道向内核发出中断申请，设置中断源的优先级其实就是设置中断通道的优先级。优先级是通过 NVIC的中断优先寄存器 NVIC_IP进行设置的。STM32只使用了NVIC_IP中的4位，最多只可以配置16种优先级，即**抢占优先级和子优先级的数量是由一个4位的数字来决定**。

优先级设定：NVIC->IPRx

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
用于表达优先级				未使用，读回为0			

STM32中断优先级分组

有5组优先级分配方式

优先级分组	抢占优先级取值范围	子优先级取值范围
NVIC_PriorityGroup_0	0	0~15
NVIC_PriorityGroup_1	0~1	0~7
NVIC_PriorityGroup_2	0~3	0~3
NVIC_PriorityGroup_3	0~7	0~1
NVIC_PriorityGroup_4	0~15	0

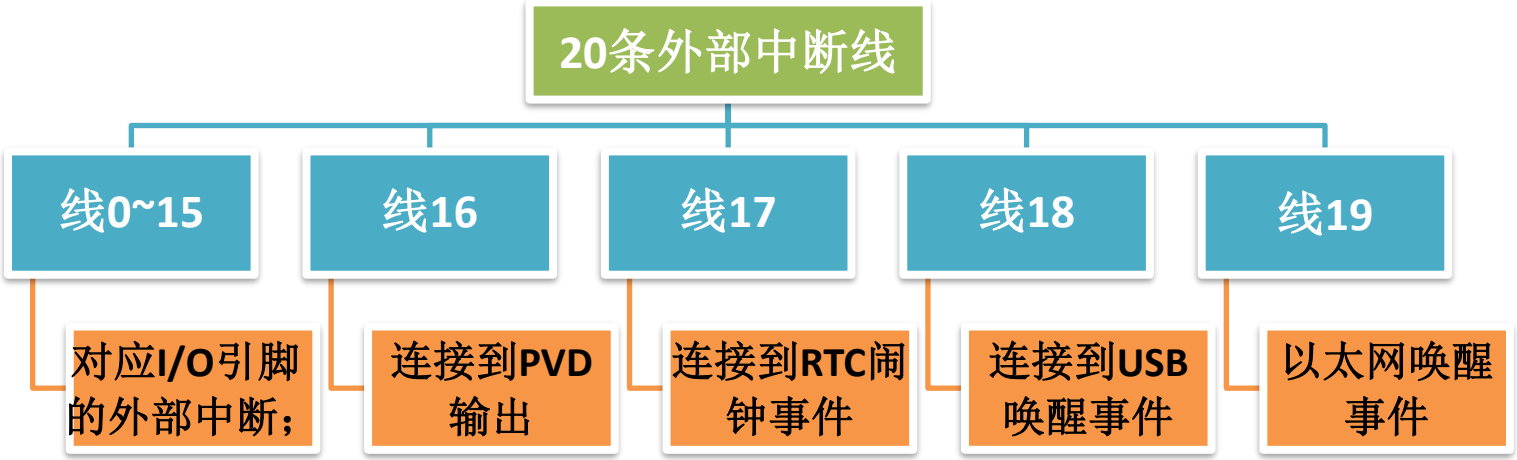
03

外部中断

外部中断EXTI

STM32 的所有 GPIO都可以配置为外部中断，用来捕捉外部信号，可以配置为下降沿中断、上升沿中断和双边沿中断。

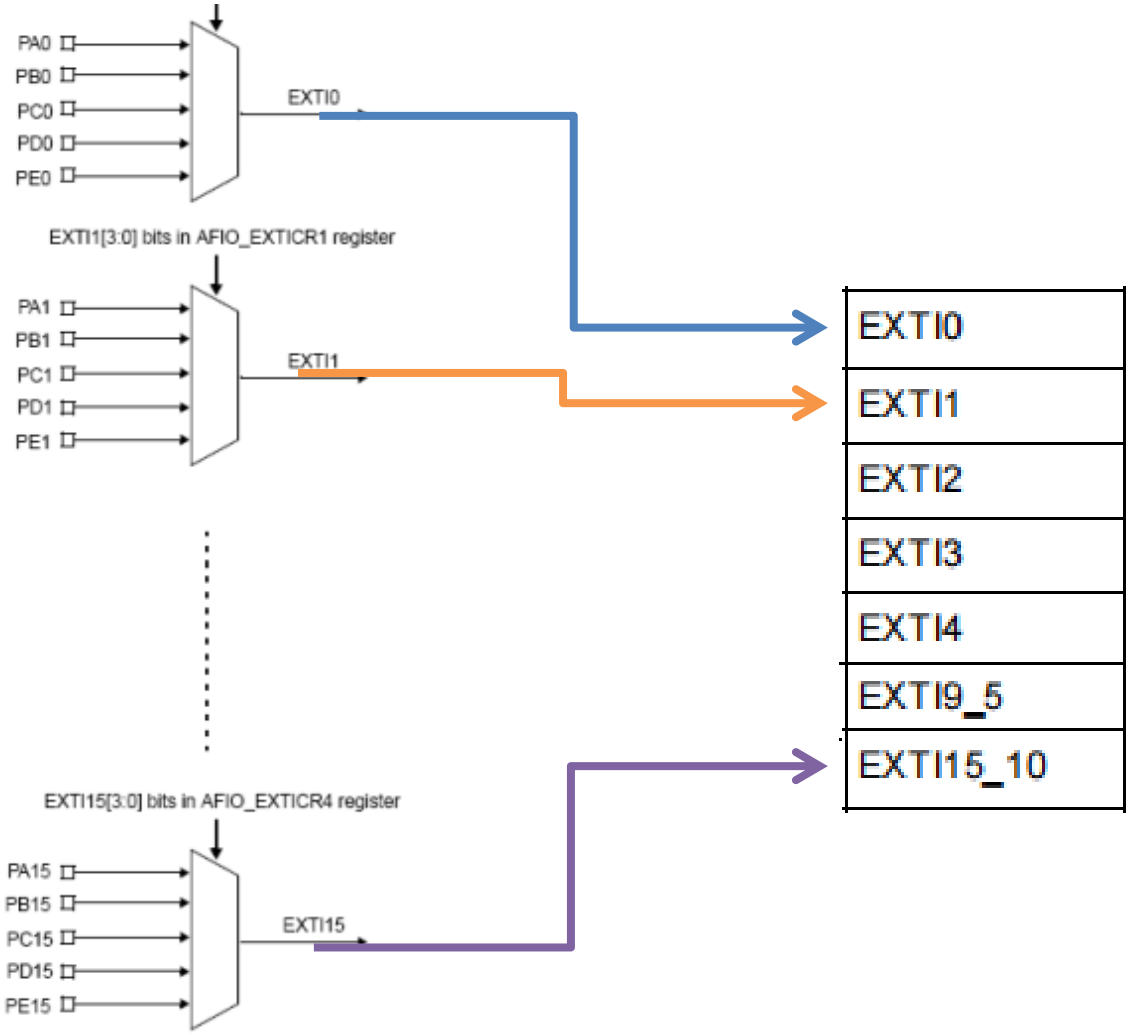
EXTI（External interrupt/event controller，外部中断/事件控制器）用来管理外部中断。



外部中断EXTI

思考：STM32供GPIO口使用的中断线只有16条，但是STM32的GPIO口远远大于16个，STM32是怎么把16个中断线和GPIO口一一对应起来的呢？

外部中断EXTI


PA0
PB0
PC0
PD0
PE0
EXTI0
EXTI1[3:0] bits in AFIO_EXTICR1 register
PA1
PB1
PC1
PD1
PE1
EXTI1
EXTI15[3:0] bits in AFIO_EXTICR4 register
PA15
PB15
PC15
PD15
PE15
EXTI15
EXTI0
EXTI1
EXTI2
EXTI3
EXTI4
EXTI9_5
EXTI15_10

04

EXTI模块的HAL库接口函数及应用

EXTI的HAL库接口函数

GPIO_Practice

stm32f1xx_hal_exti.h | stm32f1xx_hal_exti.c

- stm32f1xx_hal_tim_ex.c
- stm32f1xx_hal.c
- stm32f1xx_hal_rcc.c
- stm32f1xx_hal_rcc_ex.c
- stm32f1xx_hal_gpio.c
- stm32f1xx_hal_dma.c
- stm32f1xx_hal_cortex.c
- stm32f1xx_hal_pwr.c
- stm32f1xx_hal_flash.c
- stm32f1xx_hal_flash_ex.c
- stm32f1xx_hal_exti.c
- cmsis_armcc.h
- cmsis_compiler.h
- cmsis_version.h
- core_cm3.h
- stddef.h
- stdint.h
- stm32_hal_legacy.h
- stm32f103xe.h
- stm32f1xx.h
- stm32f1xx_hal.h
- stm32f1xx_hal_conf.h
- stm32f1xx_hal_cortex.h
- stm32f1xx_hal_def.h
- stm32f1xx_hal_dma.h
- stm32f1xx_hal_dma_ex.h
- stm32f1xx_hal_exti.h
- stm32f1xx_hal_flash.h
- stm32f1xx_hal_flash_ex.h

```
276   * @{
277   */
278 /* Configuration functions ****************************************************/
279 HAL_StatusTypeDef HAL_EXTI_SetConfigLine(EXTI_HandleTypeDef *hexti, EXTI_ConfigTypeDef *pExtiConfig);
280 HAL_StatusTypeDef HAL_EXTI_GetConfigLine(EXTI_HandleTypeDef *hexti, EXTI_ConfigTypeDef *pExtiConfig);
281 HAL_StatusTypeDef HAL_EXTI_ClearConfigLine(EXTI_HandleTypeDef *hexti);
282 HAL_StatusTypeDef HAL_EXTI_RegisterCallback(EXTI_HandleTypeDef *hexti, EXTI_CallbackIDTypeDef CallbackID, void (*pPendingCbfn)(void));
283 HAL_StatusTypeDef HAL_EXTI_GetHandle(EXTI_HandleTypeDef *hexti, uint32_t ExtiLine);
284 /**
285   * @}
286   */
287
288 /** @defgroup EXTI_Exported_Functions_Group2 IO operation functions
289   * @brief    IO operation functions
290   * @{
291   */
292 /* IO operation functions *****************************************************/
293 void              HAL_EXTI_IRQHandler(EXTI_HandleTypeDef *hexti);
294 uint32_t          HAL_EXTI_GetPending(EXTI_HandleTypeDef *hexti, uint32_t Edge);
295 void              HAL_EXTI_ClearPending(EXTI_HandleTypeDef *hexti, uint32_t Edge);
296 void              HAL_EXTI_GenerateSWI(EXTI_HandleTypeDef *hexti);
297
298 /**
299   * @}
300   */
301
302 /**
303   * @}
304   */
```

EXTI的HAL库接口函数的源码在源文件stm32f1xx_hal_exti.c中，其对应的头文件stm32f1xx_hal_exti.h声明了EXTI所有的库函数，共8种。

EXTI的HAL库接口函数

EXTI的HAL库接口函数的源码定义在stm32f1xx_hal_exti.c源文件中，其对应的头文件stm32f1xx_hal_exti.h声明了EXTI所有的库函数

类型	函数及功能描述
引脚功能操作函数	void HAL_EXTI_IRQHandler(EXTI_HandleTypeDef *hexti); 功能描述：外部中断EXTI的中断处理函数
	uint32_t HAL_EXTI_GetPending(EXTI_HandleTypeDef *hexti, uint32_t Edge); 功能描述：获取被挂起的外部中断线/事件线
	void HAL_EXTI_ClearPending(EXTI_HandleTypeDef *hexti, uint32_t Edge); 功能描述：清除被挂起的外部中断/事件线
	void HAL_EXTI_GenerateSWI(EXTI_HandleTypeDef *hexti); 功能描述：产生一个软件中断

EXTI的HAL库接口函数

EXTI的HAL库接口函数的源码定义在stm32f1xx_hal_exti.c源文件中，其对应的头文件stm32f1xx_hal_exti.h声明了EXTI所有的库函数

类型	函数及功能描述
配置函数	HAL_EXTI_SetConfigLine(EXTI_HandleTypeDef *hexti, EXTI_ConfigTypeDef *pExtiConfig); 功能描述：设置外部中断EXTI的外部中断线路
	HAL_EXTI_GetConfigLine(EXTI_HandleTypeDef *hexti, EXTI_ConfigTypeDef *pExtiConfig); 功能描述：获取已配置好的外部中断EXTI的外部中断线路
	HAL_StatusTypeDef HAL_EXTI_ClearConfigLine(EXTI_HandleTypeDef *hexti); 功能描述：将已设置好的外部中断线路清除
	HAL_EXTI_GetHandle(EXTI_HandleTypeDef *hexti, uint32_t ExtiLine); 功能描述：获取EXTI的句柄

EXTI的HAL库接口函数

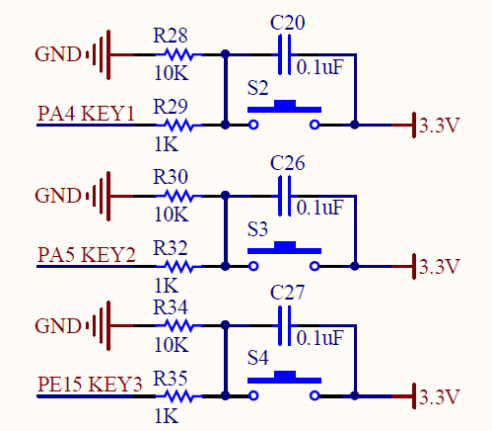
项目	函数说明
函数原型	**void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)**
功能描述	作为所有外部中断发生后的通用处理函数
入口参数	**GPIO_Pin**：连接到对应外部中断线的引脚，范围是**GPIO_PIN_0~GPIO_PIN_15**
返回值	无
注意事项	（1）所有外部中断服务程序均调用该函数完成中断处理 （2）函数内部根据**GPIO_Pin**的取值判断中断源，并清除对应外部中断线的中断标志 （3）函数内部调用外部中断回调函数**HAL_GPIO_EXTI_Callback( )**完成实际的处理任务 （4）该函数由**CubeMX**自动生成

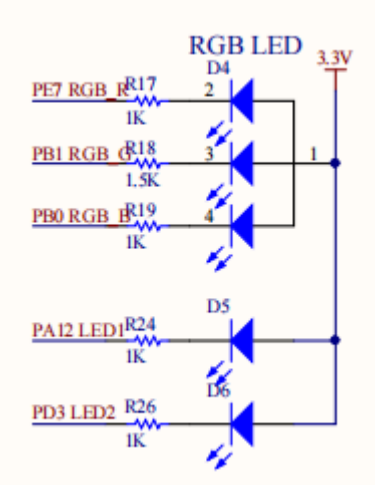
EXTI的HAL库接口函数

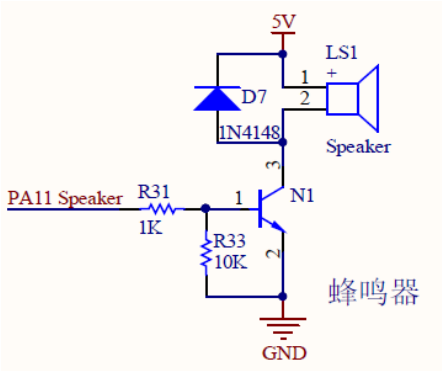
项目	说函数明
函数原型	**void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin);**
功能描述	外部中断回调函数，用于处理具体的中断服务
入口参数	**GPIO_Pin**：连接到对应外部中断线的引脚，范围是**GPIO_PIN_0~GPIO_PIN_15**
返回值	无
注意事项	（1）该函数由外部中断通用处理函数**HAL_GPIO_EXTI_IRQHandler( )**调用，完成所有外部中断的任务处理。 （2）函数内部先根据**GPIO_Pin**的取值来判断中断源，然后执行对应的中断任务 （3）该函数由用户根据实际需求编写

EXTI的HAL库应用实例

硬件设计







功能

主函数实现RGB灯按1秒间隔循环闪烁。通过外部中断实现按下按键KEY1，LED1绿灯亮5秒后熄灭；按下按键KEY2， LED2绿灯亮5秒后熄灭；按下按键KEY3，蜂鸣器响5秒停止。（观察当按键按下时，RGB灯的状态反应）

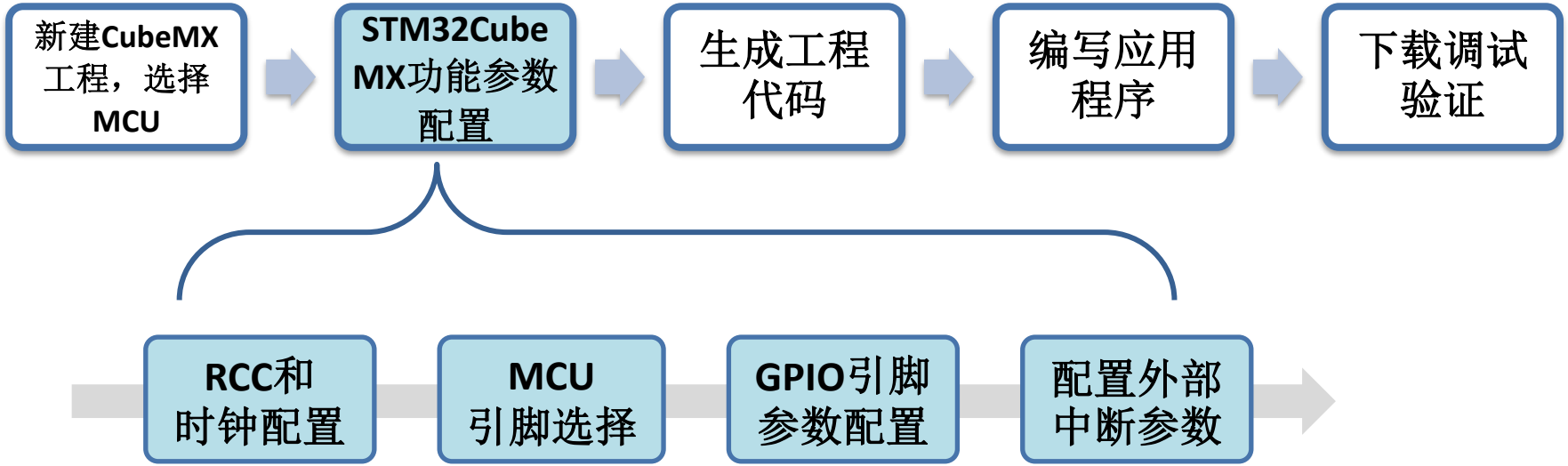
EXTI的HAL库应用实例

优先级分组选择NVIC_PriorityGroup_2，三个按键的外部中断优先级设置如下表所示。

中断源	抢占优先级	响应优先级
EXIT_Line4 interrupt（KEY1）	**2**	**1**
EXIT_Line5 interrupt（KEY2）	**2**	**0**
EXIT_Line15 interrupt（KEY3）	**1**	**0**

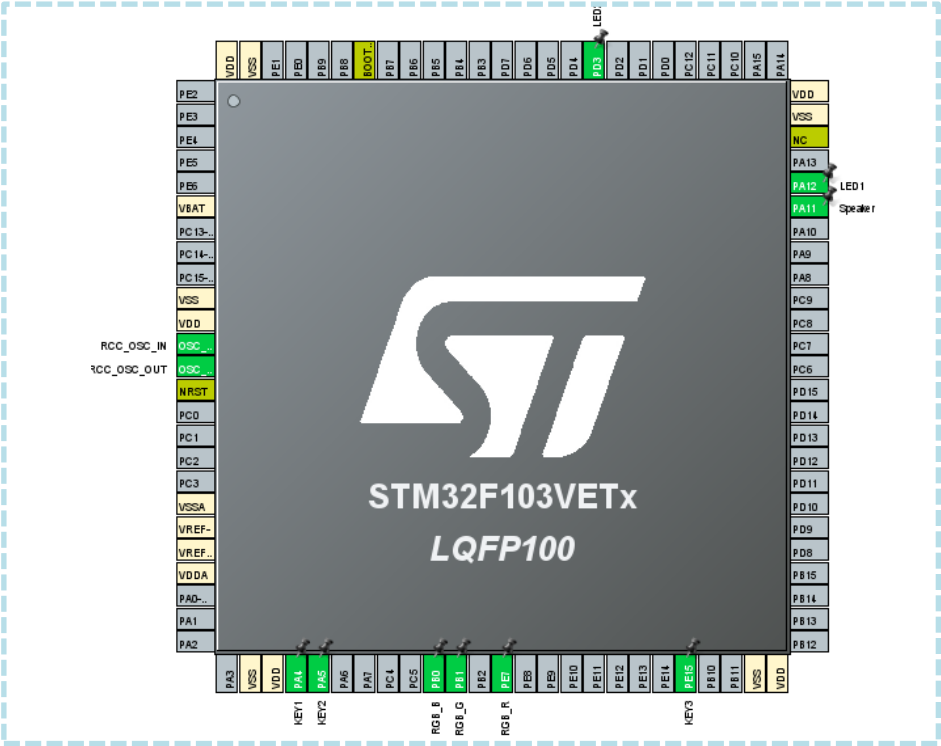
EXTI的HAL库应用实例

软件设计流程



软件设计——新建STM32CubeMX工程，选择MCU

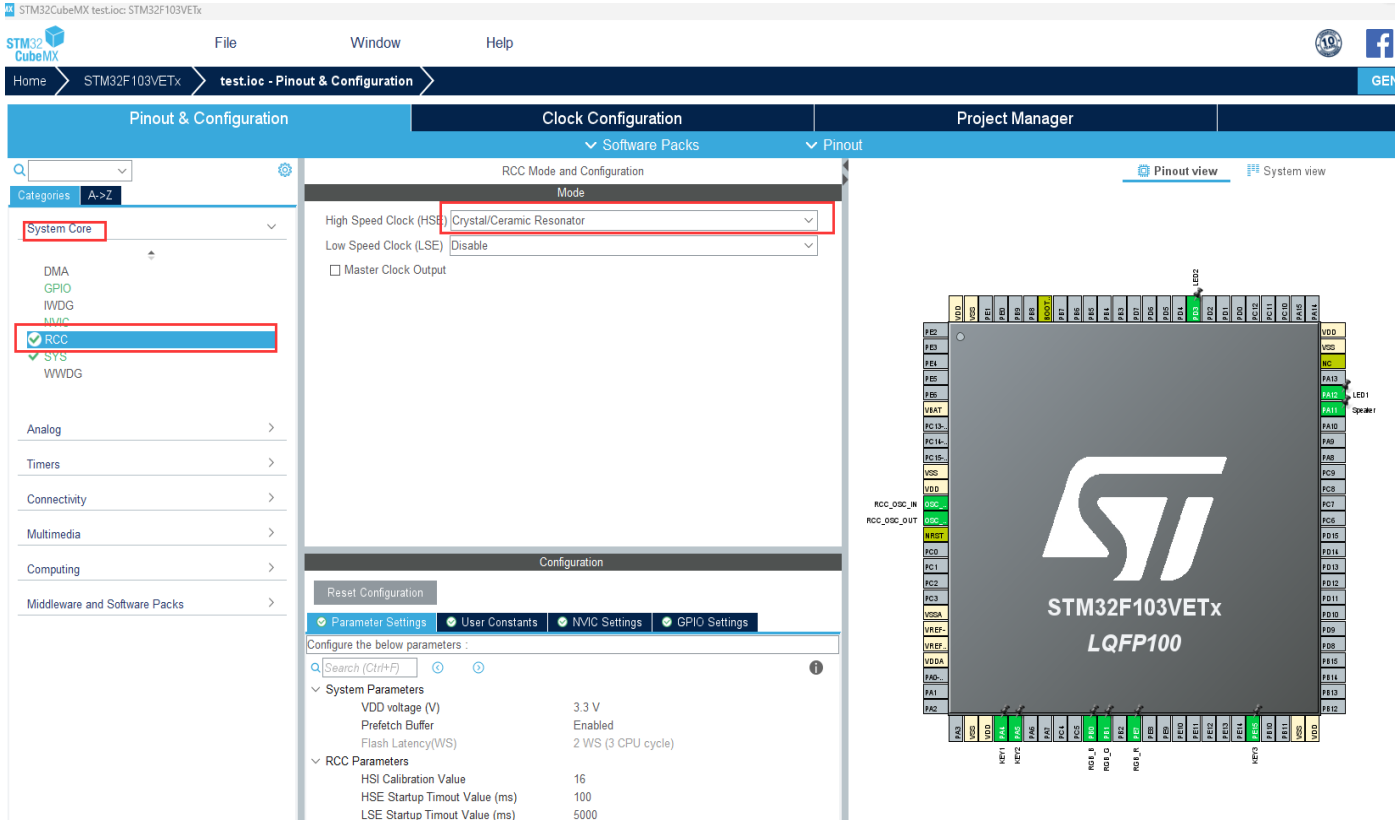
在D盘或其他盘符目录下新建一个文件夹，用来存放后面建立的CubeMX工程。需要注意的是：所建工程文件名最好是英文名称，且最好是英文路径。这里选用**STM32F103VET6**芯片。



软件设计——STM32CubeMX功能参数配置

RCC配置

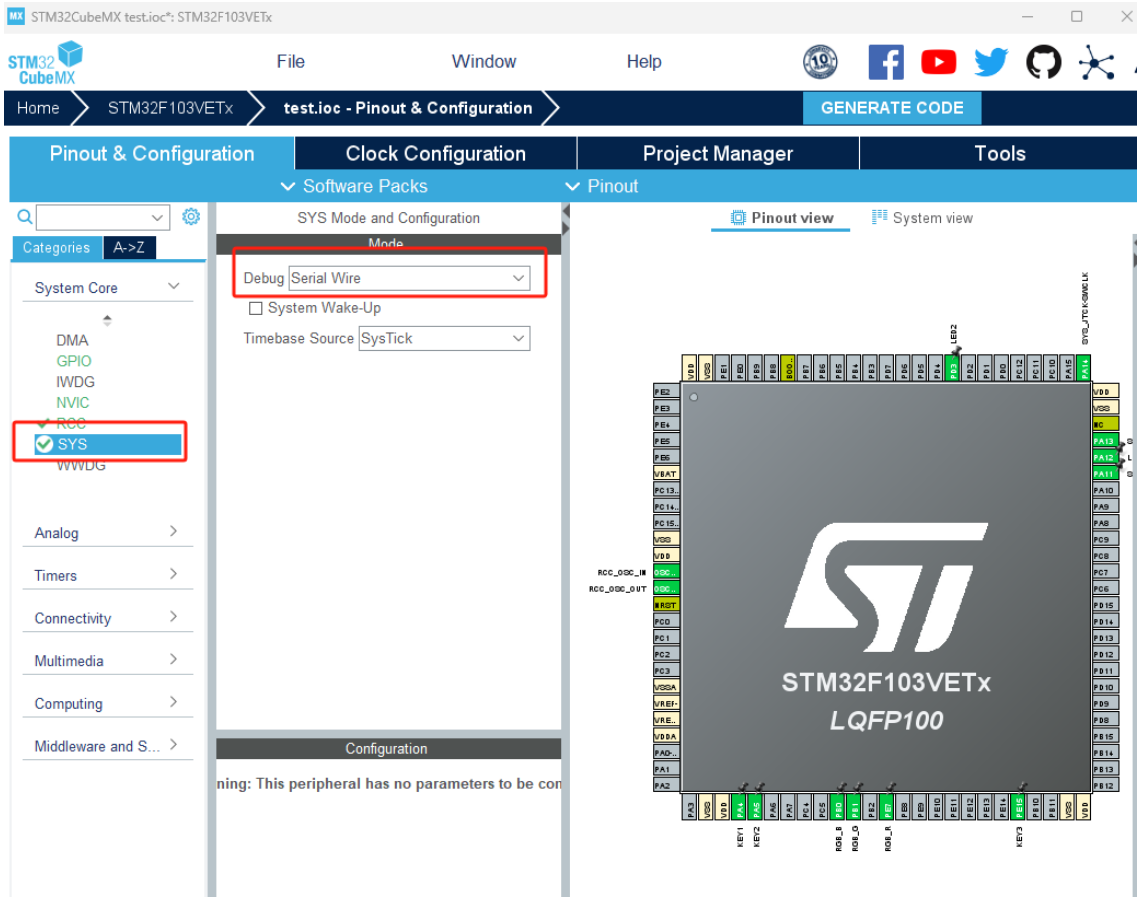
HSE选择“Crystal/Ceramic Resonator”（晶振/陶瓷谐振器），LSE选择“Disable”



软件设计——STM32CubeMX功能参数配置

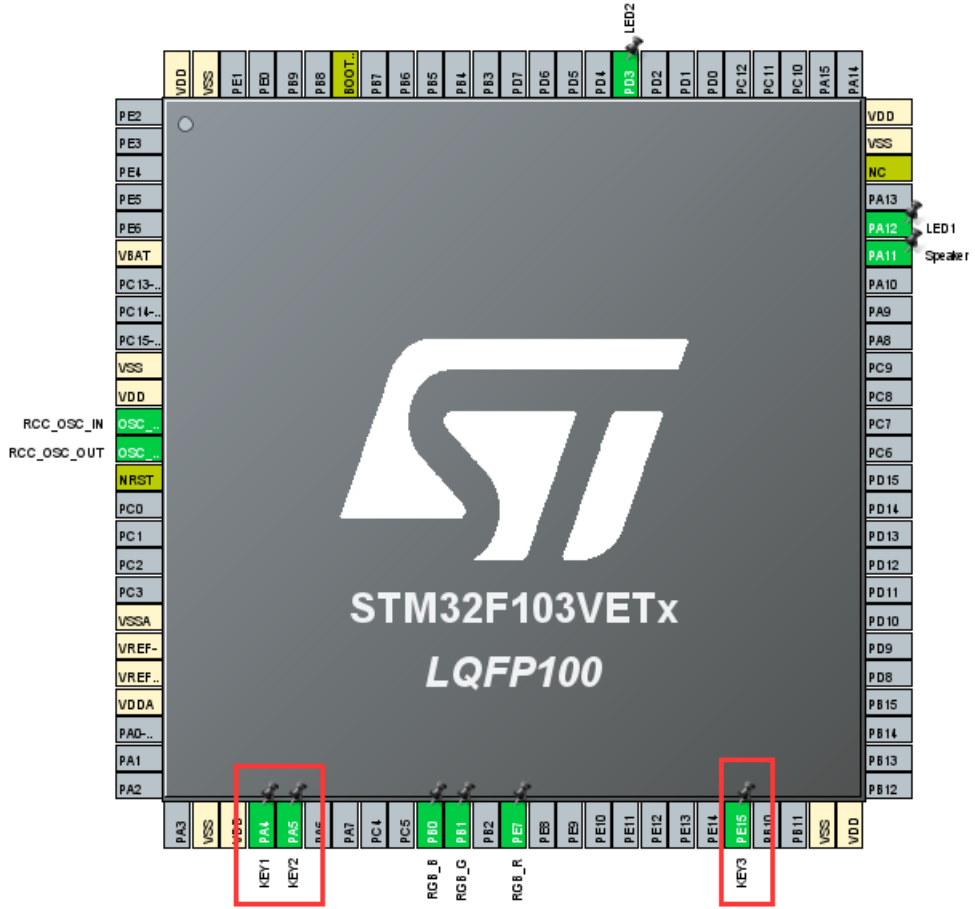
SYS配置

SYS模块用于配置程序的调试接口，这里Debug选择”Serial Wire”



软件设计——STM32CubeMX功能参数配置

MCU引脚选择



- 设置PA4为GPIO_EXTI4外部中断线，作为KEY1按键使用，用于控制LED1绿灯亮灭；
- 设置PA5为GPIO_EXTI5外部中断线，作为KEY2按键使用，用于控制LED2绿灯亮灭；
- 设置PE15为GPIO_EXTI15，作为按键KEY3使用，用于控制蜂鸣器Speaker响或不响。

软件设计——STM32CubeMX功能参数配置

GPIO引脚参数配置

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Software Packs | Pinout

Categories | A->Z

System Core
- DMA
- GPIO
- IWDG
- NVIC
- RCC
- SYS
- WWDG

Analog
Timers
Connectivity
Multimedia
Computing
Middleware and Software Packs

GPIO Mode and Configuration

Configuration

Group By Peripherals

GPIO | RCC | NVIC

Search Signals

Search (Ctrl+F)

Show only Modified Pins

Pin ...	Signal o...	GPIO output...	GPIO mode	GPIO Pull-up/Pull...	Maximum output s...	User Label	Modified
PA4	n/a	n/a	External Interrupt Mode with Rising edge trigger det...	Pull-down	n/a	KEY1	☑
PA5	n/a	n/a	External Interrupt Mode with Rising edge trigger det...	Pull-down	n/a	KEY2	☑
PA11	n/a	Low	Output Push Pull	No pull-up and no...	Low	Speaker	☑
PA12	n/a	High	Output Push Pull	No pull-up and no...	Low	LED1	☑
PB0	n/a	High	Output Push Pull	No pull-up and no...	Low	RGB_B	☑
PB1	n/a	High	Output Push Pull	No pull-up and no...	Low	RGB_G	☑
PD3	n/a	High	Output Push Pull	No pull-up and no...	Low	LED2	☑
PE7	n/a	High	Output Push Pull	No pull-up and no...	Low	RGB_R	☑
PE15	n/a	n/a	External Interrupt Mode with Rising edge trigger det...	Pull-down	n/a	KEY3	☑

PA4 Configuration :

GPIO mode: External Interrupt Mode with Rising edge trigger detection

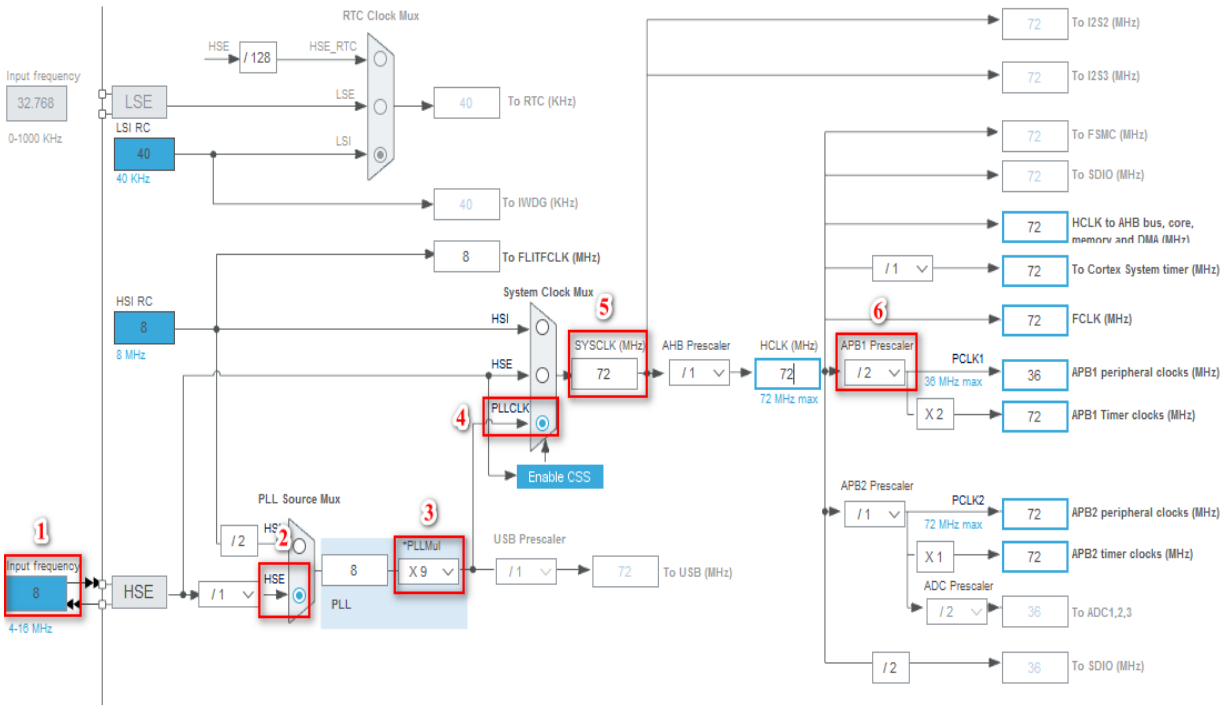
GPIO Pull-up/Pull-down: Pull-down

User Label: KEY1

在“GPIO”参数配置中设置PA4、PA5、PE15为上升沿触发（也可配置为下降沿触发），添加用户标签KEY1、KEY2、KEY3。

软件设计——STM32CubeMX功能参数配置

时钟配置



系统时钟采用外部高速时钟，配置STM32F103系列芯片最大时钟为72MHz，配置APB2为72MHz，配置APB1为36MHz。

软件设计——STM32CubeMX功能参数配置

配置外部中断参数

Pinout & Configuration | Clock Configuration

Software Packs | Pinout

Categories | A->Z

System Core
- DMA
- GPIO
- IWDG
- NVIC
- RCC
- SYS
- WWDG

Analog

Timers

Connectivity

Multimedia

Computing

Middleware and Softw...

NVIC Mode and Configuration

Configuration

NVIC | Code generation

Priority Group: 2 bits for pre-emption priority 2 bits for s...

☐ Sort by Premption Priority and Sub Priority ☐ Sort by interrupts names

Search: Search (Ctrl+F) Show: available interrupts ☑ Force DMA channels Interrupts

NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
Non maskable interrupt	☑	0	0
Hard fault interrupt	☑	0	0
Memory management fault	☑	0	0
Prefetch fault, memory access fault	☑	0	0
Undefined instruction or illegal state	☑	0	0
System service call via SWI instruction	☑	0	0
Debug monitor	☑	0	0
Pendable request for system service	☑	0	0
Time base: System tick timer	☑	0	0
PVD interrupt through EXTI line 16	☐	0	0
Flash global interrupt	☐	0	0
RCC global interrupt	☐	0	0
EXTI line4 interrupt	☑	2	1
EXTI line[9:5] interrupts	☑	2	0
EXTI line[15:10] interrupts	☑	1	0

- 在“NVIC”选项页中，设置“Priority Group”进行优先级分组；
- 勾选“EXIT_Line4 interrupt”使能PA4外部中断线；
- 类似地，勾选“EXTI_Line5 interrupt”使能PA5外部中断线；勾选“EXTI_Line5 interrupt”使能PA5外部中断线。

软件设计——生成工程代码

配置keil工程名称和存放位置

STM32CubeMX test.ioc: STM32F103VETx

File Window Help

Home > STM32F103VETx > test.ioc - Project Manager

GENERATE CODE

Pinout & Configuration | Clock Configuration | Project Manager | Tools

Project

Code Generator

Advanced Settings

Project Settings

Project Name: test

Project Location: C:\Users\syzx\Desktop\STM32CubeMX_TEST\Key_Interrupt\ Browse

Application Structure: Basic ☐ Do not generate the main()

Toolchain Folder Location: C:\Users\syzx\Desktop\STM32CubeMX_TEST\Key_Interrupt\

Toolchain / IDE: MDK-ARM Min Version: V5.32 ☐ Generate Under Root

Linker Settings

Minimum Heap Size: 0x200

Minimum Stack Size: 0x400

Thread-safe Settings

Cortex-M3NS

☐ Enable multi-threaded support

Thread-safe Locking Strategy: Default – Mapping suitable strategy depending on RTOS selection.

Mcu and Firmware Package

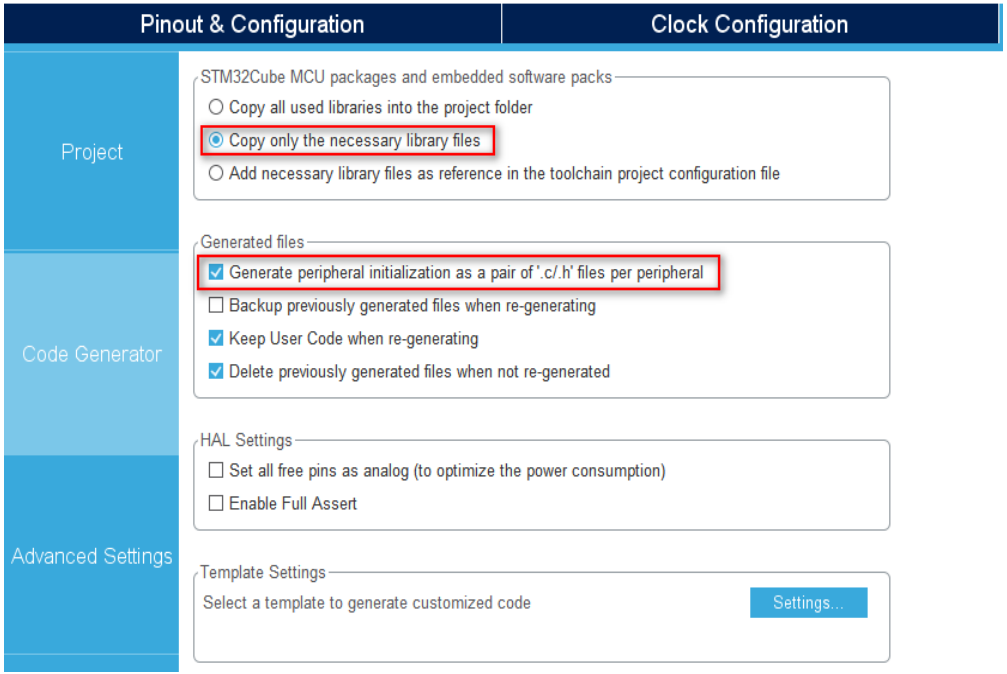
Mcu Reference: STM32F103VETx

Firmware Package Name and Version: STM32Cube FW_F1 V1.8.5 ☑ Use latest available version

☑ Use Default Firmware Location

Firmware Relative Path: C:/Users/syzx/STM32Cube/Repository/STM32Cube_FW_F1_V1.8.5 Browse

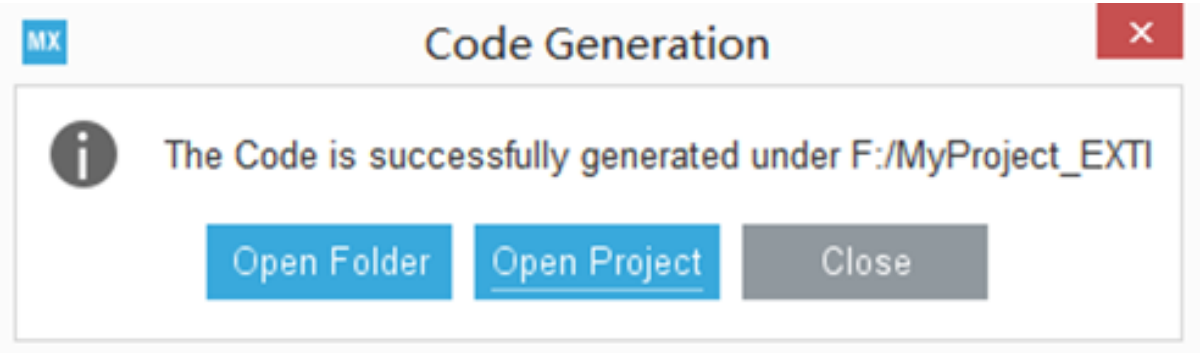
软件设计——生成工程代码



在“Code Generator”选项栏中找到“Generated files”框，勾选“Generate peripheral initialization as a pair of ’.c/.h’ files per IP”，将外设初始化的代码设置生成为独立的.c源文件和.h头文件。

软件设计——生成工程代码

通过STM32CubeMX的菜单栏中的“Generate Code”生成工程代码，生成代码后，会提示是否打开该工程窗口



软件设计——编写应用程序

- 在keil 中单击编译按钮，对生成的工程进行编译；
- 在stm32f1xx_it.c文件中可以找到所定义的中断函数；
- 外部的三个按键中断触发后，首先会调用GPIO相应引脚的外部中断处理函数。

main.c | gpio.c | stm32f1xx_it.c | stm32f1xx_hal_gpio.c | startup_stm32f103xe.s

```c
/**
  * @brief This function handles EXTI line4 interrupt.
  */
void EXTI4_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI4_IRQn 0 */

  /* USER CODE END EXTI4_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(KEY1_Pin);
  /* USER CODE BEGIN EXTI4_IRQn 1 */

  /* USER CODE END EXTI4_IRQn 1 */
}

/**
  * @brief This function handles EXTI line[9:5] interrupts.
  */
void EXTI9_5_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI9_5_IRQn 0 */

  /* USER CODE END EXTI9_5_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(KEY2_Pin);
  /* USER CODE BEGIN EXTI9_5_IRQn 1 */

  /* USER CODE END EXTI9_5_IRQn 1 */
}

/**
  * @brief This function handles EXTI line[15:10] interrupts.
  */
void EXTI15_10_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI15_10_IRQn 0 */

  /* USER CODE END EXTI15_10_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(KEY3_Pin);
  /* USER CODE BEGIN EXTI15_10_IRQn 1 */

  /* USER CODE END EXTI15_10_IRQn 1 */
}
```

软件设计——编写应用程序

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != RESET)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}
```

HAL_GPIO_EXTI_IRQHandler( )函数原型

- 比如按键KEY1连接在PA4引脚上会触发外部中断EXTI4的中断处理程序EXTI4_IRQHandler(void)，而这个函数又调用HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_4)函数，参数为GPIO_PIN_4，即EXTI4中断。单击右键跳转到这个函数，可以看到该函数的原型，

软件设计——编写应用程序

```c
/* USER CODE BEGIN 0 */
 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
 {
      if(GPIO_Pin == GPIO_PIN_4)
      {
         HAL_GPIO_WritePin(GPIOA,GPIO_PIN_12,GPIO_PIN_RESET);
         /* 延时1000毫秒 */
         HAL_Delay(5000);
         HAL_GPIO_WritePin(GPIOA,GPIO_PIN_12,GPIO_PIN_SET);
      }

  if(GPIO_Pin == GPIO_PIN_5)
  {
     HAL_GPIO_WritePin(GPIOD,GPIO_PIN_3,GPIO_PIN_RESET);
         /* 延时1000毫秒 */
     HAL_Delay(5000);
     HAL_GPIO_WritePin(GPIOD,GPIO_PIN_3,GPIO_PIN_SET);
  }
  if(GPIO_Pin == GPIO_PIN_15)
  {
     HAL_GPIO_WritePin(GPIOA,GPIO_PIN_11,GPIO_PIN_SET);
         /* 延时1000毫秒 */
     HAL_Delay(1000);
     HAL_GPIO_WritePin(GPIOA,GPIO_PIN_11,GPIO_PIN_RESET);
  }

 }
/* USER CODE END 0 */
```

- 当中断发生时，会调用**HAL_GPIO_EXTI_Callback( )函数**，但程序中只给出了一个虚函数，**需要用户添加应用程序代码来实现相关功能。**
- 在main.c文件中的/*USER CODE BEGIN 0*/和/*USER CODE END 0*/之间添加中断回调函数相应的程序代码。

软件设计——编写应用程序

- 在main.c文件中的/*USER CODE BEGIN 3*/和/*USER CODE END 3*/之间编写主程序代码，让RGB灯以1秒的间隔循环亮灭。

```
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

     HAL_GPIO_WritePin(GPIOE,GPIO_PIN_7,GPIO_PIN_RESET);
     /* PE7输出低电平，红灯亮；PE7输出高电平，红灯灭 */
     HAL_Delay(1000); /* 延时1000毫秒 */
     HAL_GPIO_WritePin(GPIOE,GPIO_PIN_7,GPIO_PIN_SET);
     HAL_Delay(1000);

     HAL_GPIO_WritePin(GPIOB,GPIO_PIN_1,GPIO_PIN_RESET);
     HAL_Delay(1000);
     HAL_GPIO_WritePin(GPIOB,GPIO_PIN_1,GPIO_PIN_SET);
     HAL_Delay(1000);

     HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0,GPIO_PIN_RESET);
     HAL_Delay(1000);
     HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0,GPIO_PIN_SET);
     HAL_Delay(1000);
  }

  /* USER CODE END 3 */
}
```

软件设计——下载调试验证

应用程序编写完毕

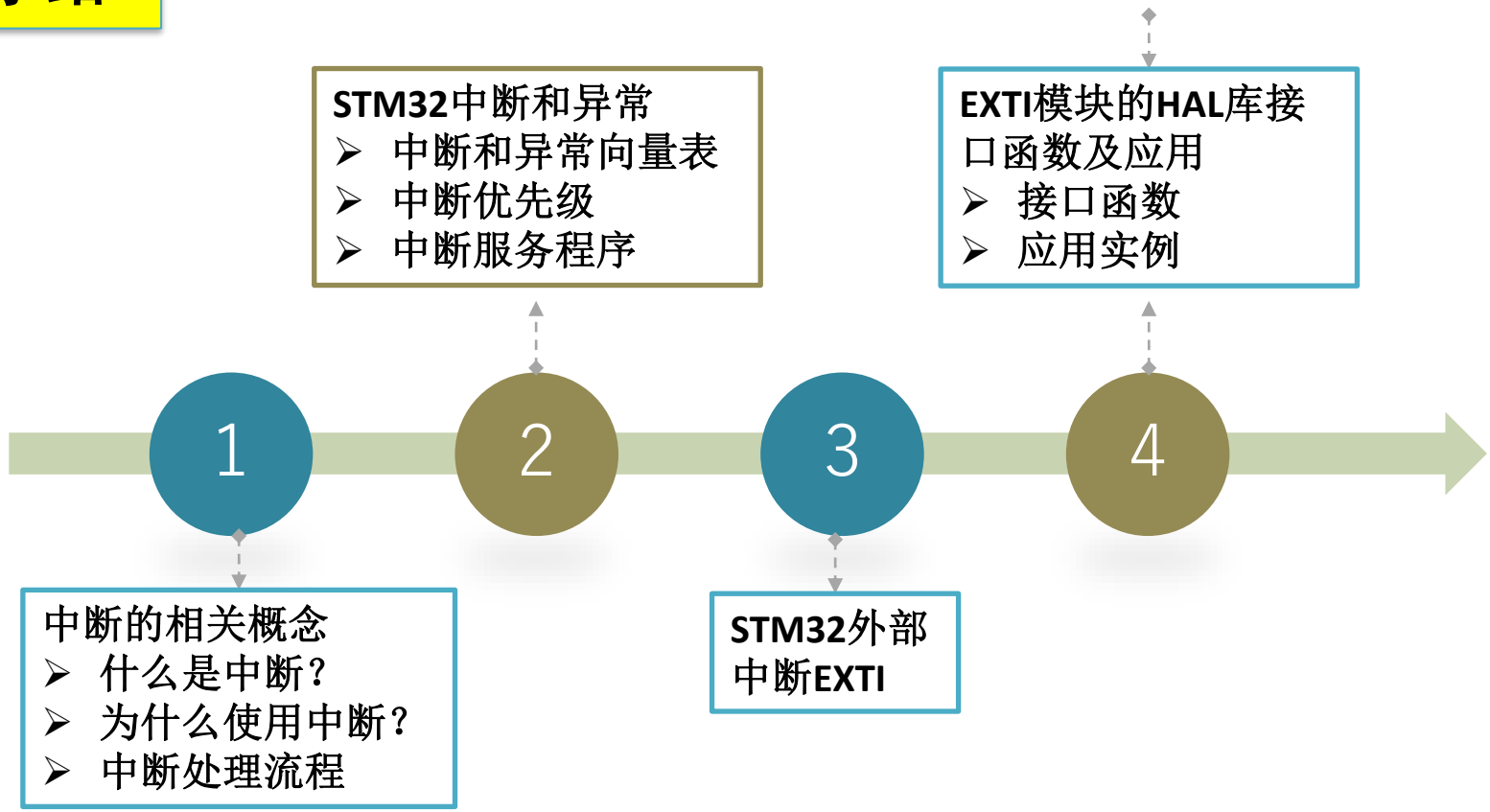
重新编译一下工程，生成工程hex文件

通过ST-LINK仿真器或其它下载器下载到目标板

进行验证

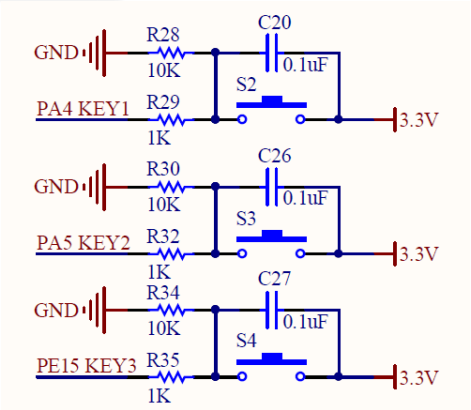
可以看到，按下相应的按键，其对应的LED灯的状态会发生改变。

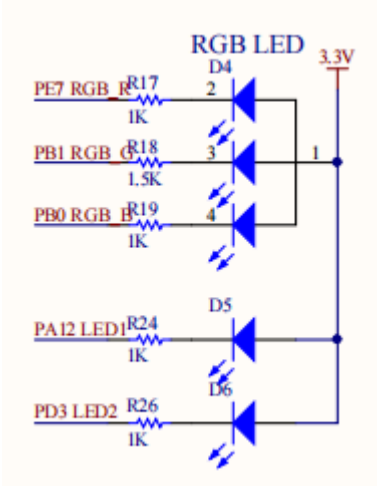
本章小结

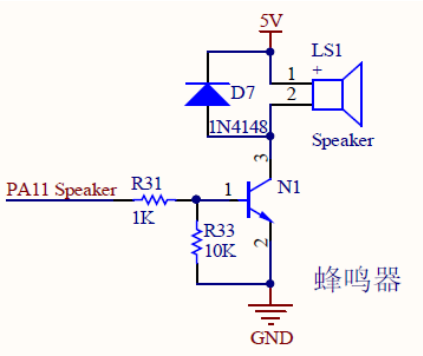

STM32中断和异常
➢ 中断和异常向量表
➢ 中断优先级
➢ 中断服务程序
EXTI模块的HAL库接口函数及应用
➢ 接口函数
➢ 应用实例
1
2
3
4
中断的相关概念
➢ 什么是中断？
➢ 为什么使用中断？
➢ 中断处理流程
STM32外部中断EXTI

实践任务

硬件设计





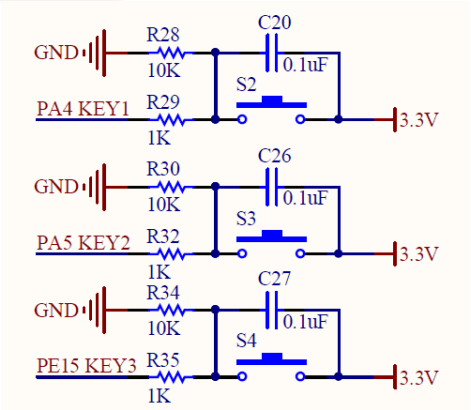


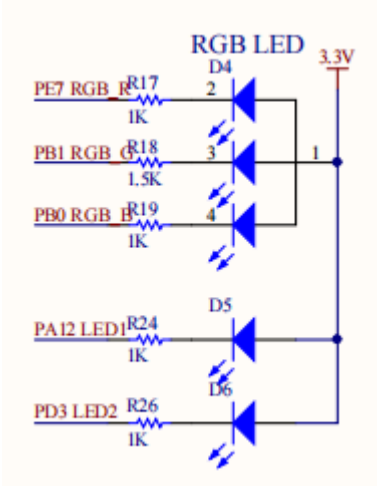
功能

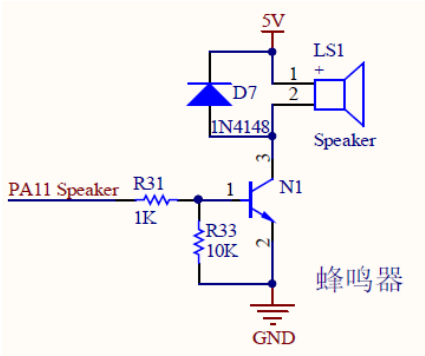
编写程序实现LED1和LED2灯处于常亮状态，当按下KEY1键时，两灯以比较慢的频率(T=1000ms)闪烁10次；当按下KEY2键时，两灯以比较快的频率(T=500ms)闪烁10次；且KEY2的抢占优先级高于KEY1，可以形成中断嵌套。

实践任务

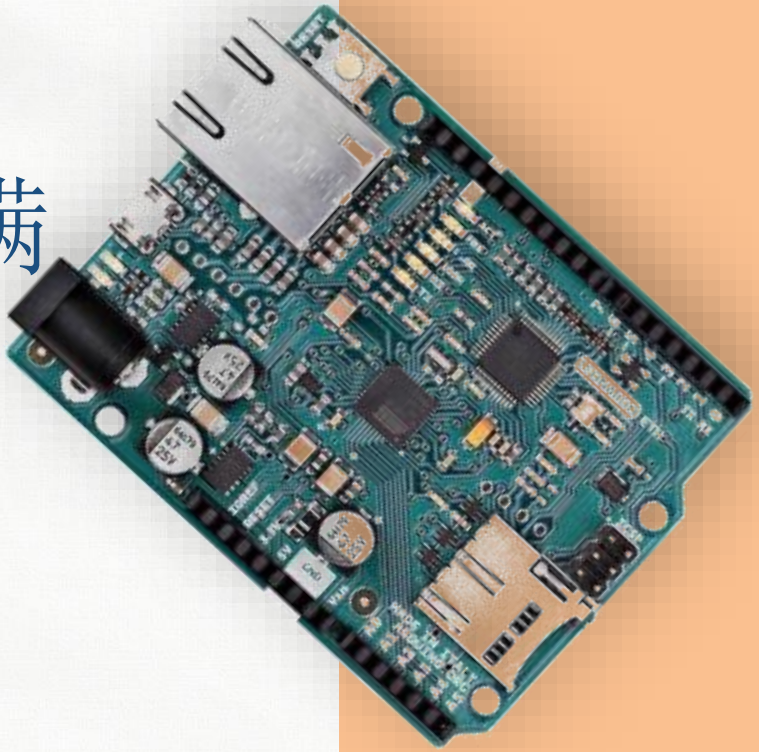
硬件设计







现象

可以观察到的现象：两个LED灯呈常亮状态，当按下KEY1键后，两灯以比较慢的频率闪烁10次；当按下KEY2键后，两灯以比较快的频率闪烁10次；如果在执行KEY1键的任务中，按下KEY2键，就会切换到KEY2键的工作频率，执行完KEY2键的任务后，再继续执行未完成的KEY1键的任务，形成中断嵌套。

本章结束，收获满满